SECURE ROM HAL

Project Showcase

Portfolio overview for GitHub, LinkedIn, and technical interviews

A compact presentation of architecture, implementation approach, verification workflow, limitations, and future improvements.

Important: host tests and RTL simulation are not physical board validation or security certification.

PROJECT SHOWCASE / 01 OF 50

Project in one sentence

A modular C/HAL and RTL learning framework that explores peripheral driver APIs, register access, boot-flow-oriented modules, and a custom AXI4-Lite control peripheral using host tests and RTL simulation.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Problem statement

Embedded software and programmable logic meet at register maps, status codes, timing, and protocol handshakes. This project turns those interfaces into a small, testable repository instead of leaving them as isolated examples.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

PROJECT SHOWCASE / 03 OF 50

Portfolio goals

The project demonstrates code organization, driver API design, negative testing, simulation modeling, RTL protocol reasoning, build automation, and documentation practices.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

PROJECT SHOWCASE / 04 OF 50

Architecture overview

Application code calls HAL APIs; the HAL uses platform register utilities or a mock backend; OSAL separates higher-level logic from runtime choices; ROM-oriented modules model boot-flow responsibilities; RTL provides a memory-mapped peripheral example.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Repository layout

app/ contains the demo entry point; platform/ holds shared utilities; hal/ contains peripheral drivers; simulation/ provides host models; osal/ separates runtime abstractions; rom/ contains boot-flow-oriented modules; rtl/ and tests/ contain verification assets.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

PROJECT SHOWCASE / 06 OF 50

Build workflow

CMake configures targets, the C compiler builds them, and CTest runs registered tests. Icarus Verilog compiles RTL testbenches and vvp executes simulations. The layers are deliberately tested separately.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Quick start

From the repository root, configure with `cmake -S . -B build`, compile with `cmake --build build`, and run `ctest --test-dir build --output-on-failure`. For RTL, use the documented Icarus commands or the verification script.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Example command

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

PROJECT SHOWCASE / 08 OF 50

C language baseline

The build declares C11. Fixed-width types and explicit status codes make hardware-facing code easier to reason about. Public functions validate inputs and report failures rather than relying on undefined behavior.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

PROJECT SHOWCASE / 09 OF 50

Platform register layer

Central register helpers make access patterns consistent and easier to mock. The host model supports logic tests; a real MMIO implementation must account for address mapping, volatile access, memory ordering, and platform-specific requirements.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

HAL design principle

A hardware abstraction layer should hide register details behind a stable API. Callers should know what operation they request and what status it returned, not every implementation-specific register offset.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

GPIO module

GPIO exercises pin validation, state updates, read/write/toggle semantics, and boundary testing. Host-side behavior should be described as simulation, not as proof that a physical board pin toggles correctly.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

PROJECT SHOWCASE / 12 OF 50

UART module

UART examples cover byte transmission, status handling, and concurrency-oriented buffering. A ring buffer separates producers from the transmit path, but real interrupt safety depends on target-specific synchronization.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

PROJECT SHOWCASE / 13 OF 50

Ring buffer

The ring buffer provides a compact exercise in wrap-around, full/empty states, and producer-consumer coordination. Tests should cover boundaries and repeated wrap cycles, not just a single enqueue/dequeue.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

SPI module

SPI APIs demonstrate initialization, byte transfer, buffer transfer, pointer validation, and busy/error reporting. A simulated device makes deterministic host tests possible without claiming electrical or sensor-level validation.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

PROJECT SHOWCASE / 15 OF 50

I2C module

The I2C abstraction and device model are useful for studying addressing, register transactions, NACK/error handling, and timeouts. Bus electrical behavior remains outside a basic host simulation.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

OSAL layer

The OS abstraction layer gives code a common interface for bare-metal and RTOS-style behavior. Portability requires documenting blocking semantics, timing units, ISR safety, and differences between backends.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Boot flow

The ROM-oriented modules organize firmware image handling, integrity/signature-related checks, and boot-flow decisions. The exact security properties depend on the implementation and must be reviewed before making strong claims.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Integrity versus authenticity

Checksums and hashes can detect data changes, but they do not authenticate the source by themselves. A production secure boot design needs trusted cryptographic verification, key management, rollback policy, and a threat model.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Example command

```
grep -nEi 'PASS|FAIL|ERROR|FATAL' reports/final_verification.log
```

PROJECT SHOWCASE / 19 OF 50

AXI4-Lite peripheral

The custom RTL exposes CONTROL, STATUS, and OUTPUT registers. It is a focused example of a memory-mapped control peripheral and a software/RTL interface contract.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Register map

The described offsets are CONTROL at 0x0, STATUS at 0x4, and OUTPUT at 0x8. The 8-bit output and control/status behavior should be confirmed against the exact RTL and register header before use.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Write channel

AXI4-Lite address and data arrive on independent channels. The RTL tracks pending information so that AW and W can be accepted in different cycles, then returns a response.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Read channel

The read path accepts an address and returns data plus a response. RVALID/RDATA/RRESP must remain stable while the receiver applies backpressure.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Byte strobes

Write strobes select valid byte lanes. Testing selected and unselected lanes is important because software and RTL must agree on which bits change during partial writes.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Invalid access behavior

The design describes decode errors for unmapped offsets and writes to the read-only STATUS register. Negative tests should verify both the response and the absence of unintended state changes.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

PROJECT SHOWCASE / 25 OF 50

Functional RTL verification

The functional testbench exercises normal bus operations and expected register behavior. A simulation should report explicit pass/fail outcomes and stop with a nonzero status on failure.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Backpressure verification

Backpressure tests hold READY low and check that a response remains stable until accepted. This targets a common protocol corner case that happy-path tests often miss.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Reset verification

Reset tests check that register state returns to expected defaults. More complete coverage can include reset while address, data, or response handshakes are pending.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Example command

```
iverilog -g2012 -o build/verification/axi_reset rtl/axi_secure_peripheral.v rtl/tb/tb_axi_reset.v
vvp build/verification/axi_reset
```

C driver layer

The AXI driver translates API calls into register operations, validates arguments, tracks initialization, and masks output data to the supported width. A host mock tests software logic, not a physical AXI connection.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Negative tests

Negative tests exercise null arguments, invalid state, unsupported operations, and other error paths. These tests help show that the API fails predictably instead of crashing or silently continuing.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

PROJECT SHOWCASE / 30 OF 50

CTest

CTest gives a repeatable summary of registered C test executables. Test registration must be checked explicitly; a built binary is not automatically part of the regression suite.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

PROJECT SHOWCASE / 31 OF 50

Icarus Verilog

Icarus compiles RTL and testbench sources; vvp runs the generated simulation. Compilation and functional pass are separate outcomes and both should be recorded.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

One-command verification

The project includes a script intended to configure, build, run C tests, compile RTL testbenches, run simulations, and write a log. Review the actual log and exit code before claiming success.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Log review

Search logs for PASS, FAIL, ERROR, and FATAL, then inspect the tail and each test stage. Printed failure text can be missed by shell automation unless the testbench returns a nonzero exit code.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Example command

```
./scripts/run_verification.sh
```

Git hygiene

Ignore build directories and generated simulator outputs, run `git diff --check`, and review `git status --short` before committing. Keep the repository focused on source, tests, scripts, and useful documentation.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Reproducibility

A useful verification record names the tested commit, tool versions, commands, and pass/fail summary. This makes the project easier for reviewers to reproduce.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

What is validated

The intended evidence is host-side C compilation/tests and RTL simulation. Those checks can validate logic and interface expectations within their test scope.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

What is not validated

No physical PS-to-PL integration, board-level electrical behavior, FPGA timing closure, or security certification should be claimed unless separately implemented and measured.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Design trade-offs

Small modules are easier to test, but host mocks simplify real hardware. A clear interface contract and a list of simulation boundaries help reviewers understand both the value and the limitations.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Potential improvement: testbench exits

Add `$fatal` or equivalent failure propagation for every failed expectation, and ensure the simulator process exits nonzero when a test fails. This lets shell automation detect functional failures.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Potential improvement: coverage

Add transaction coverage for AW-first, W-first, simultaneous AW/W, mapped/unmapped offsets, byte strobes, response backpressure, and reset in each relevant state.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Potential improvement: hardware backend

A target-specific MMIO backend could connect the driver to a real address map. This would require platform configuration, memory mapping, access ordering, and board-specific validation.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Potential improvement: cryptographic design

If genuine secure boot is a goal, integrate a reviewed cryptographic library, define the trust anchor and key lifecycle, implement rollback policy, and add known-answer and mutation tests.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

How to reproduce

Clone the repository, inspect README and CMakeLists.txt, configure and build, run CTest, then compile and run each RTL testbench. Finish by running the automation script and reviewing the generated log.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Example command

```
git status --short
git diff --check
git log --oneline -5
```

Troubleshooting: build

For a missing header, inspect include paths and capitalization. For an undefined reference, confirm the source file contains a definition and is included in the target. Read the first meaningful error before the final build-system summary.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Troubleshooting: RTL

For a simulation hang, identify the first VALID/READY handshake that does not complete. Add bounded waits and print channel signals on timeout. Check reset, response readiness, and testbench drive timing.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Troubleshooting: register mismatch

Compare the RTL offset, C header, driver, and mock. Confirm reset values, write strobes, response codes, and output masking. A mismatch between mock and RTL can allow software tests to pass while the hardware model rejects the operation.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Interview discussion

Explain the HAL/OSAL separation, register access strategy, AXI VALID/READY handshakes, independent AW/W channels, backpressure, negative tests, and why simulation does not equal hardware validation.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?

Resume-ready summary

Built a modular Embedded C/HAL learning framework with platform utilities, OS abstraction, host tests, and a custom AXI4-Lite RTL peripheral. Added functional, backpressure, and reset testbench flows plus CMake/CTest-based verification automation.

Evidence / reviewer lens

Validation note: separate source-level reasoning from observed test output.

Public release checklist

Publish only public-safe source. Remove secrets and proprietary content, run tests from a clean checkout, confirm logs, include limitations, and ensure the README commands match the current repository.

Evidence / reviewer lens

Evidence to attach: repository tree, README, and tested commit hash.

Closing summary

The value of this project is the integration of software interfaces, register-level design, RTL protocol behavior, and repeatable tests. Its strongest portfolio presentation is honest about what has been simulated and what remains future work.

Evidence / reviewer lens

Reviewer lens: what concrete interface or failure mode does this page explain?